AGH

WEAIiIB ISI

Lab 1: easyAI – Nim i Nimby

Przedmiot: Inteligencja obliczeniowa w analizie danych cyfrowych

Autorzy:	Mikołaj Dudkiewicz Eliza Petrycka
Grupa:	[6]

19 marca 2026

Spis treści

1 Opis zadania i gry **1**

2 Implementacja **1**

3 Metodologia eksperymentów **1**

4 Wyniki – zadania na 4 punkty **2**

5 Wyniki – zadania na 6 punktów **2**

6 Wyniki – zadania na 8 punktów **3**

7 Napotkane problemy **4**

8 Reprodukowanie wyników **4**

1 Opis zadania i gry

W ramach laboratorium zrealizowano wszystkie punkty z pliku `EasyAI.pdf`. Wybrana gra probabilistyczna to **Nimby**. Zakres obejmuje:

- implementację wariantu probabilistycznego gry,
- turnieje AI vs AI z kontrolą gracza rozpoczynającego,
- porównanie algorytmów Negamax (z i bez odcięcia alfa-beta),
- pomiar czasu podejmowania decyzji,
- implementację wariantu expecti-minimax z odcięciem alfa-beta.

Gra bazowa to klasyczny Nim (wariant misere): gracz zabierający ostatni element przegrywa. Stan początkowy w eksperymentach: trzy stosy $(3, 4, 5)$. Wariant Nimby wprowadza szum wykonania: dla ruchu "zabierz k" (gdy $k > 1$), z prawdopodobieństwem 0.1 wykonywane jest "zabierz $k - 1$".

2 Implementacja

Kod znajduje się w pliku `lab_01/nimby_experiments.py`. Zaimplementowano:

- klasę `NimbyGame` (deterministyczny Nim i probabilistyczny Nimby przez parametr `error_probability`),
- `NegamaxNoAB` – Negamax bez odcięcia alfa-beta,
- `ExpectiNegamaxAB` – wariant expecti-minimax z odcięciem alfa-beta,
- `TimedAIPlayer` – zliczanie czasu decyzji i liczby ruchów,
- runner eksperymentów zapisujący wyniki do `lab_01/results.json`.

Ważny aspekt implementacyjny. W środowisku probabilistycznym nie można tracić spójności między `make_move` a `unmake_move`. Dlatego rzeczywista liczba zabranych elementów jest zapisywana na stosie historii i cofana 1:1. Dla expecti-minimax zamiast losowania użyto jawnych węzłów szansy (normalny ruch vs "slip") i wartości oczekiwanej:

$$E = (1 - p)\, V_{\text{normal}} + p\, V_{\text{slip}}, \quad p = 0.1.$$

Wykresy do raportu są generowane automatycznie przez skrypt `lab_01/generate_plots.py` na podstawie `lab_01/results.json`.

3 Metodologia eksperymentów

Wszystkie testy uruchomiono skryptem:

```
uv run python lab_01/nimby_experiments.py
```

Wspólne ustawienia:

- naprzemienne rozpoczynanie partii (eliminacja biasu pierwszego ruchu),
- limit 200 półruchów na partię (w praktyce brak remisów),
- te same stany początkowe i kontrola ziaren losowości po numerze partii.

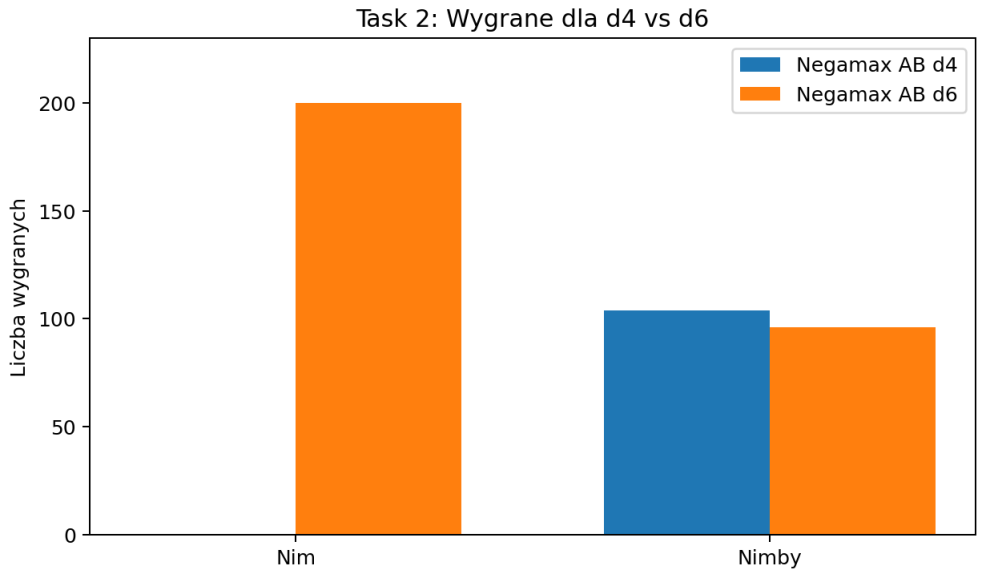
4 Wyniki – zadania na 4 punkty

Porównano dwa ustawienia głębokości Negamax: `depth=4` i `depth=6`, osobno dla Nim (deterministyczny) i Nimby (probabilistyczny).

Tabela 1: NegamaxAB depth 4 vs depth 6 (200 partii)

Wariant	Wygrane d4	Wygrane d6	Remisy	Średni czas ruchu [ms] d4 : d6
Nim (det.)	0	200	0	0.7146 : 5.4021
Nimby (prob.)	104	96	0	0.8147 : 6.0359

Komentarz. W wariancie deterministycznym głębsze przeszukiwanie dominuje całkowicie. W wariancie probabilistycznym różnica głębokości przestaje być monotonicznie korzystna: przypadkowość wykonania ruchu zaburza planowanie klasycznego Negamax.



Rysunek 1: Task 2: liczba wygranych dla NegamaxAB d4 i d6.

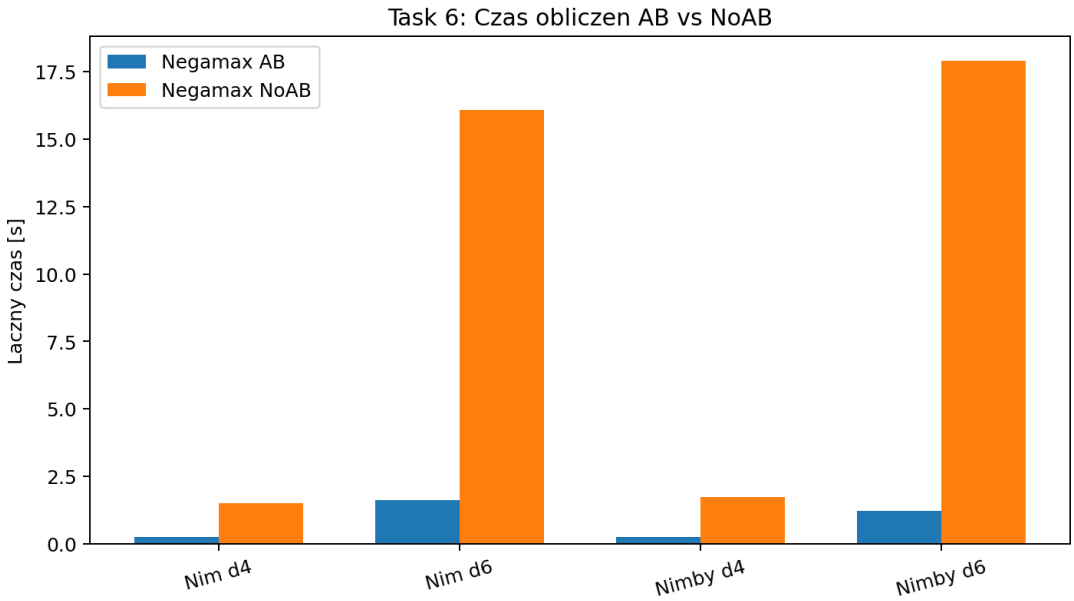
5 Wyniki – zadania na 6 punktów

Wykonano porównanie Negamax z i bez odcięcia alfa-beta, dla dwóch głębokości (4, 6) i dla obu wariantów gry.

Tabela 2: Negamax z/bez alfa-beta (100 partii na każde starcie)

Wariant	Starcie	Wygrane A	Wygrane B	Czas A [s]	Czas B [s]
Nim	AB d4 vs NoAB d4	50	50	0.3694	2.1376
Nim	AB d6 vs NoAB d6	50	50	2.3406	23.1431
Nimby	AB d4 vs NoAB d4	59	41	0.3349	2.3374
Nimby	AB d6 vs NoAB d6	51	49	1.8052	25.6892

Wnioski. Decyzje strategiczne AB i NoAB są porównywalne (w Nim idealnie identyczne), ale wydajność czasowa AB jest wielokrotnie lepsza. Dla `depth=6` bez odcięcia koszt obliczeń jest rzędu 10× większy.



Rysunek 2: Task 6: porównanie łącznego czasu obliczeń AB vs NoAB.

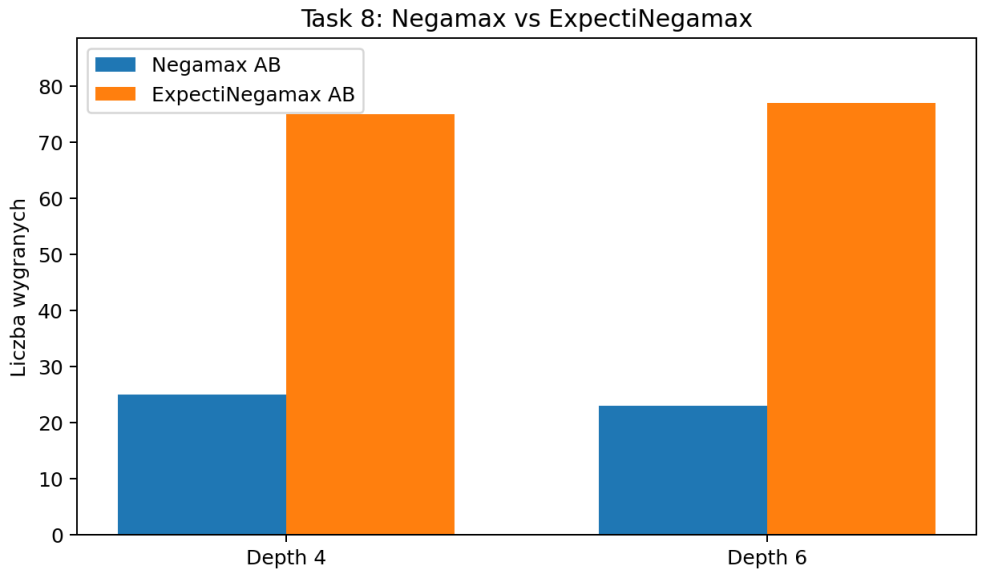
6 Wyniki – zadania na 8 punktów

Zaimplementowano expecti-minimax z odcięciem alfa-beta i porównano z klasycznym NegamaxAB w Nimby.

Tabela 3: NegamaxAB vs ExpectiNegamaxAB (Nimby, 100 partii)

Starcie	Wygrane Negamax	Wygrane Expecti	Czas Negamax [s]	Czas Expecti [s]
Depth 4	25	75	0.3544	0.9316
Depth 6	23	77	2.1210	11.0509

Wnioski końcowe. Expecti-minimax wyraźnie poprawia skuteczność w grze probabilistycznej, bo modeluje ryzyko błędu wykonania ruchu. Cena za tę poprawę to wyższy czas obliczeń (około 2.6× dla depth 4 i ponad 5× dla depth 6).



Rysunek 3: Task 8: NegamaxAB kontra ExpectiNegamaxAB (Nimby).

7 Napotkane problemy

Głównym problemem była zgodność logiki probabilistycznej z mechanizmem przeszukiwania drzewa. Negamax zakłada spójność wykonania/cofania ruchu; przy naiwnej losowości w `make_move` można łatwo uszkodzić stan gry. Problem rozwiązano przez:

- jawną rejestrację rzeczywiście wykonanego ruchu,
- deterministyczne cofanie ruchów po historii,
- osobne modelowanie węzłów szansy w expecti-minimax.

8 Reprodukowanie wyników

- generowanie wykresów: `uv run python lab_01/generate_plots.py`
- uruchomienie: `uv run python lab_01/nimby_experiments.py`
- dane wyjściowe: `lab_01/results.json`
- raport: `lab_01/report.tex`